

MiniGrid 空間導航與探索模擬實驗： 基於 CNN 與 LSTM 的深度強化學習

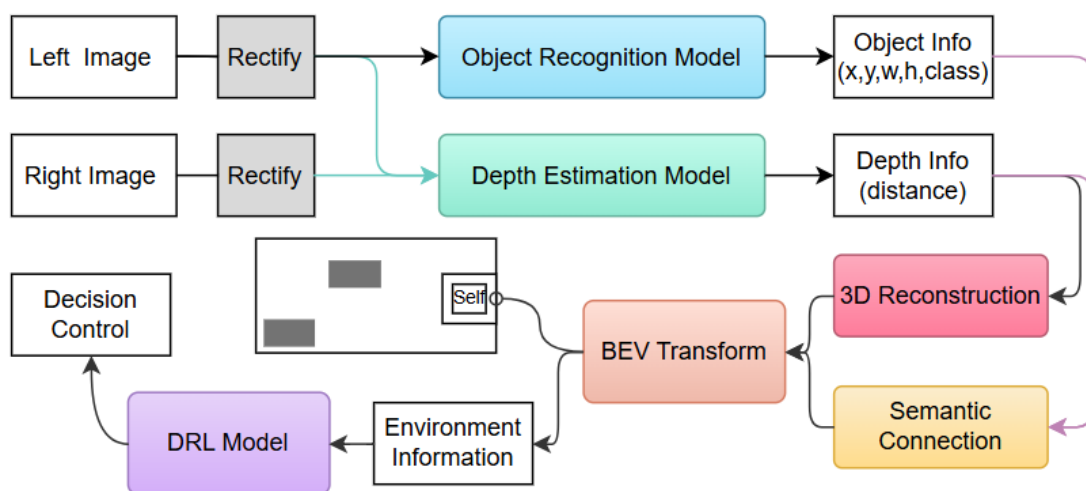
MiniGrid Spatial Navigation and Exploration Experiment:
CNN-LSTM Based Deep Reinforcement Learning

目次

介紹&動機.....	3
Enviroment.....	4
Reward.....	5
Resward (Re-Designed).....	5
Deep Q Learning.....	6
Introduction.....	6
Why does DQN not work?.....	6
Deep Reinforcement Learning.....	7
Deep Learning Models.....	8
Convolutional Neural Network for Actor–Critic.....	8
Long Short-Term Memory for Actor–Critic.....	8
Program Architecture.....	9
Experiment.....	10
CNN Policy & PPO.....	10
CNN+LSTM Policy & PPO.....	10
Display & Demo.....	11
結語.....	12

介紹&動機

強化學習演算法被認為能夠像人類一樣透過反覆試驗來學習複雜的適應性行為，應對不可預測的環境、複雜的動態，並透過優化行動來最大化獎勵，從而完成組裝或導航等高精度任務。這項專題報著重在對中小型機器載具或自主系統上對於空間探索、行為、動作任務、邏輯判斷為主要研究目標，其想法源於近期個人的一項關於立體視覺系統於機器自主系統的實驗擴展，期望通過由雙目立體視覺估測與視覺辨識產生的 Bird Eye View 資訊，建立一套基於強化學習的演算法模型，使系統能夠在空間中進行空間探索並自動化獨自完成任務，整體架構如下圖所展示為理想設計的整體架構，利用有限的視野資訊作為強化學習模型的輸入並對採取的動作進行規劃。



現階段的侷限在於欲希望此系統的自主決策與規劃缺乏三維重建的資訊來作為訓練環境、且立體視覺系統目前所生成的 BEV 資訊目前準確度不夠高且雜訊過多，因此轉往嘗試以目前現有的模擬環境來實驗與學習強化學習方面的知識並建構雛形實驗，像使用 StanfordVL 的 GIBSON 模擬由感測器掃描過的真實環境，但考量到我的算力資源相當有限制，改往找尋低算力即可以模擬自主系統空間探索與任務執行的模擬環境，並最終選擇 OpenAI 的 Minigrid 生成環境作為本次實驗環境。本次實驗方向旨在探索深度學習模型與強化學習算法的整合發展，通過實驗常見的深度學習模型架構探討其在空間探索與智慧決策的學習泛化性與實用效果。

專案套件：**PyTorch**、Minigrid API、gym、numpy、imageio

Resource：<https://github.com/StanfordVL/GibsonEnv>

Resource：<https://minigrid.farama.org/index.html>

Enviroment

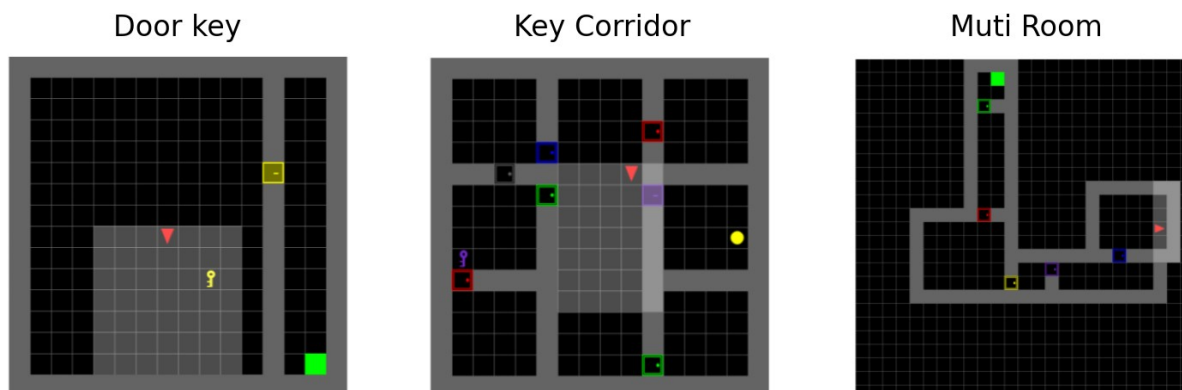
Partially Observable Markov Decision Process, POMDP Enviroment

觀測空間：部分觀測（agent 只能看到附近格子）

動作空間：離散動作（左轉 0、右轉 1、前進 2、拿起 3、放下 4、啟用對象 5、done 6）

任務 Door Key、Key Corridor、Multi Room

訓練環境：隨機生成，測試環境集：固定 seed 生成 N 張



Minigrd Agent 視野資訊：ImgObsWrapper 抓取資訊為 $7*7*3$ 的 RGB Array。

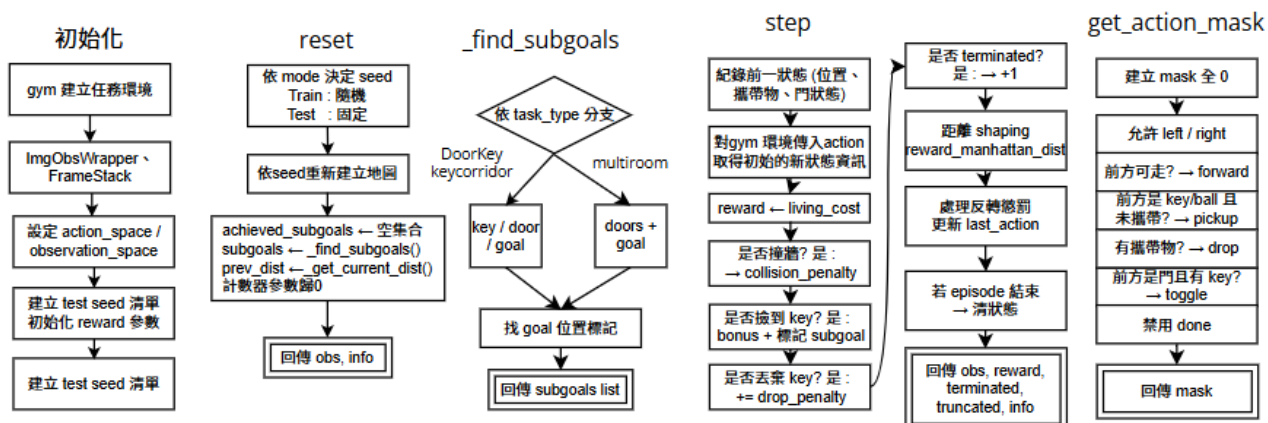
為部分可觀察馬可夫決策過程，模擬代理人決策程序是假設系統動態由 MDP 決定，但是代理人無法直接觀察目前的狀態，它必須要根據模型的全域與部分區域觀察結果來推斷狀態的分佈，這樣也更符合中小型自動機器與自主系統在實際應用中的情況。

DoorKey：需在環境中搜尋鑰匙，完成多階段行為需求到達目標位置。

MultiRoom：多間互相連結的房間組成網格世界，Agent 需穿過一系列房間方可到達終點。

KeyCorridor：需撿起任務指定物件，鑰匙與目標物件分佈於不同需探索房間中。

實驗方式：訓練階段通過隨機產生環境地圖使 Agent 在其地圖上採樣並學習，訓練後通過固定 Seed 生成 100 張固定的測試地圖並計算通過率與測試策略表現評估。



Enviroment Class 結構與函數

Reward

* $\text{Minigrid default Reward} = 1 - 0.9 * (\text{step_count} / \text{max_steps})$

實驗 Door Key 、Multi Room 初期時，發現嘗試訓練 DQN、Actor-critic 約 3000~5000 Epochs 都無法在測試集上成功一次，考慮到是因為預設的 minigrid 環境給予 reward 為時間懲罰式的稀疏獎勵，完成任務才能拿到正回報，成功到達 goal 時給 $1 - 0.9 * (\text{step_count} / \text{max_steps})$ 的 reward，這導致真正有效訓練樣本極少。

Reward (potential-based reward shaping)

加入 reward 調整靠近 goal 給小正 reward，設定係數 0.01~0.1，引導 agent 先往 goal 方向走，順便探索物件或其他區域，參考 Ng, Harada, and Russell (1999) 的 Potential-Based Reward Shaping。

Resward (Re-Designed)

隨著時間的進行我發現，會出現 Agent 在原點不動避免損失的方式，由於加入時間罰項時，導致 PPO 的 advantage 計算偏負，value function 低估移動價值，政策崩潰到不動以最小化損失。

使重新設計的 Reward 符合這些原則：

- * **任務導向性**：獎勵對應任務的關鍵事件。
- * **階段性引導**：依智能體當前狀態動態調整子目標。
- * **行為約束**：透過懲罰項目抑制無效或破壞性行為。
- * **穩定學習**：避免過大或過頻獎勵，降低策略震盪風險。

Reward Design :

每個時間步給予固定負獎勵：Reward = -0.002

當智能體嘗試向前移動但前方為不可重疊物件(ex.牆壁)：Reward = -0.05

當智能體首次成功拾取鑰匙時 Reward = +0.5 (使 Agent 能夠有效學習互動物件方式)

鑰匙丟棄懲罰：Reward = -0.1 懲罰以避免策略中出現不合理的狀態回退行為

開門獎勵：Reward = +0.5

終點獎勵：Reward = +1.0

引入 **Manhattan Distance-based Reward Shaping** 並依智能體狀態動態設定子目標：

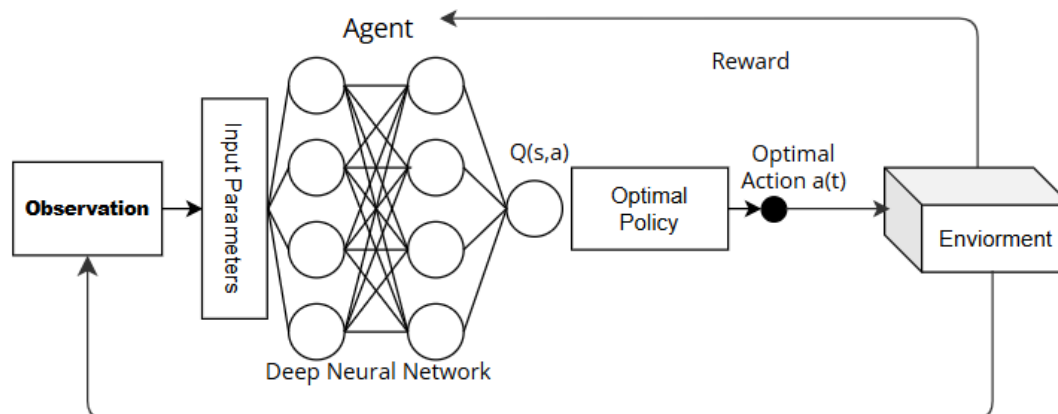
$R_{\text{shape}} = 0.02 * (d_{\text{prev}} - d_{\text{curr}})$ ，結合事件型獎勵與距離型 shaping。

後續根據實驗遇到的問題，額外加入機制若是重複轉向 3、4 次以上就給予負值 reward。

Deep Q Learning

Introduction

發現到 Q-Learning 的 Q table 是在 state 與 action 有限不多的情況下才可能被建立起來並有效的使用，並不能有效解決所有問題，當環境是變化性的、廣大且多種可能性，無法有效使用 Q learning，因此改用類神經網路架構提取 feature 並逼近 Q function 對當下適合的 action 做出預測。在 MiniGrid 中，用 ImgObsWrapper 抓取資訊為 $7*7*3$ 的 RGB Array，由於對 Agent 的輸入視野是一張圖像，為了留住關聯、位置等特徵資訊，使用 Convolutional 神經網路 CNN 建模，輸出為連續空間的函數近似的 Q value function，並使用 ϵ -greedy policy。訓練探索階段使用 ϵ 機率亂選 action，訓練前期 ϵ 大多隨機，後期 ϵ 小越來越貪婪收斂模型，每一筆神經網路的訓練資料使用 (state, action, reward, next_state, done) 為一組，DQN 內用 TD error 修正 $Q(s,a)$ ，將 TD target Q 值作為模型訓練的擬合輸出資訊，reward + next state Q 值組合而成的 TD target 作為 Label，並使用 MSE 計算預測 Q 值與 TD target 的差距。



Why does DQN not work?

嘗試在 DoorKey、MultiRoom 訓練後發現，更新 5,000,000 Step 後都無法對 DQN 模型進行合適的改進，分析認為 DQN 為擅於反應式決策狀態幾乎是 **Fully Observable**，在 minigrid 中可能與環境互動但環境狀態已經改變，環境本質上違反 DQN 的 MDP 假設，DQN 可能被迫學習同一場景對應多個 Q 值，可能出現情況是相同走廊畫面，但來自不同位置、面向不同方向時畫面旋轉但語意不同等情況不足以構成完整狀態，違反 DQN 所需的 Markov 性質，缺乏結構性探索、TD target 不穩定導致收斂困難。

經過資料搜尋，DQN 在某些受限條件下仍可運作，改良方式包括堆疊多幀 observation Frame stacking、或是優化 Double DQN (DDQN)、在 Q 網路中加入 LSTM 的 DRQN 等，或是嘗試針對因而，考量轉向嘗試其他深度強化學習演算法，Actor-Critic 或是基於 Policy Gradient 的 PPO。

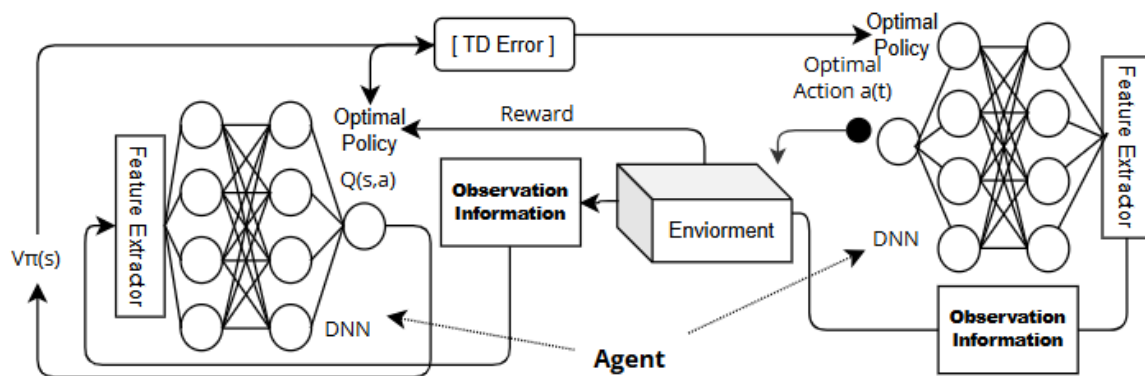
Deep Reinforcement Learning

Actor-Critic 方法概述

Actor (策略網路)：直接輸出動作機率分佈 $\pi_{\theta}(a|s)$ ，決定 agent 採取哪個動作。

Critic (價值網路)：估計狀態值 $V_{\phi}(s)$ 或行動價值 $Q_{\phi}(s,a)$ ，作為 Actor 更新的基準。

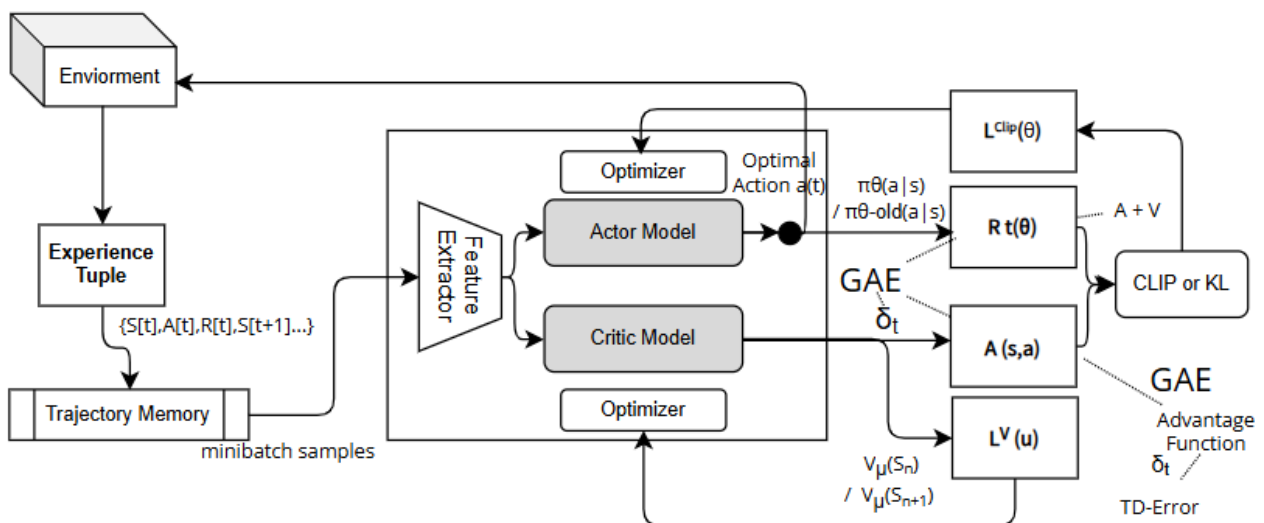
$\theta \leftarrow \theta + \alpha \nabla_{\theta} \log \pi_{\theta}(a|s) \cdot A(s,a)$ ：其中 $A(s,a)=R-V_{\phi}(s)$ 為 **Advantage** 函數，用 Critic 的估計減少策略梯度的方差，提高學習穩定性。Policy network (ex. 策略梯度方法) 輸入 state 預測每個 Action 的機率與 Value Network(ex. DQN) 預測 State Value 都是在強化學習的兩種方式，Actor-Critic 將兩著優點結合，使得模型既可以訓練連續型動作，且可以以單步更新的方式進



行，使得在訓練上可以更有效率。

PPO (Proximal Policy Optimization) 方法概述

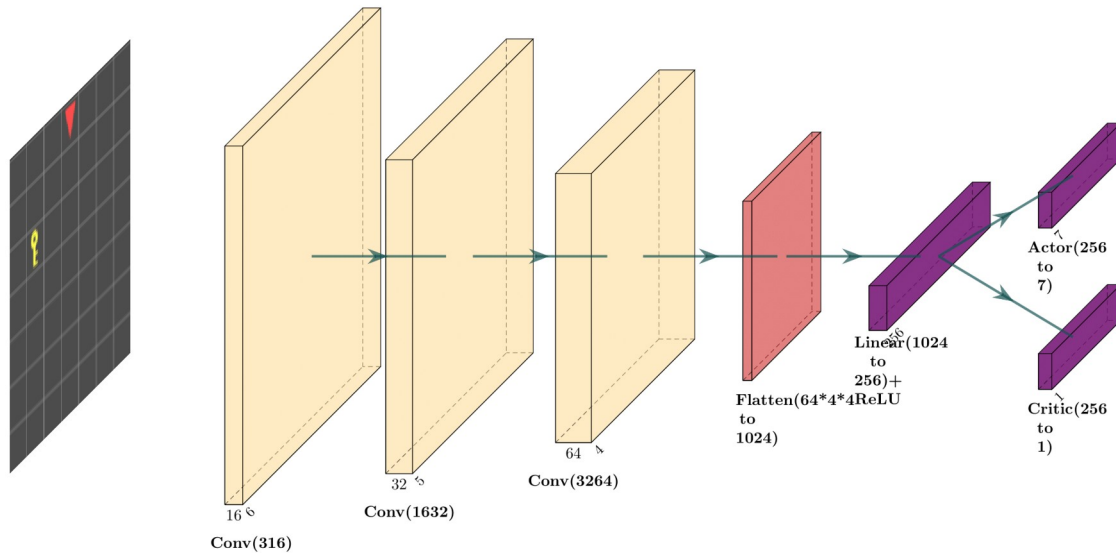
PPO 與深度學習網路整合：Actor 策略網路負責輸出行動策略 $\pi_{\theta}(a|s)$ 決定在某個狀態下要採取什麼行為，其特色在於**限制策略更新幅度**以提升穩定性以 Clip objective、KL penalty 更新 Actor 的策略，透過多次小步更新，避免策略劇烈改變導致性能崩潰，保留 Actor-Critic 架構，但在策略梯度計算上增加了**剪裁與正則化機制**，使訓練更穩定，此方法能有效處理部分可觀測與長期依賴問題，因此在 MiniGrid 這類環境中是更適合的選擇。



Deep Learning Models

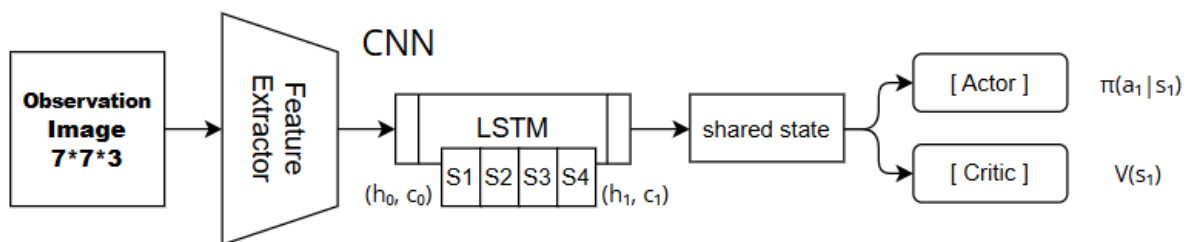
Convolutional Neural Network for Actor-Critic

觀測以局部視覺圖像 $7 \times 7 \times 3$ 的格子視窗形式產生，二維空間結構數據 具有顯著的空間相關性與局部模式包含牆壁、門、鑰匙的位置，透過卷積核與池化操作能有效提取空間特徵並保存局部結構，同時參數共享降低模型複雜度，有利於從環境畫面學習狀態資訊。



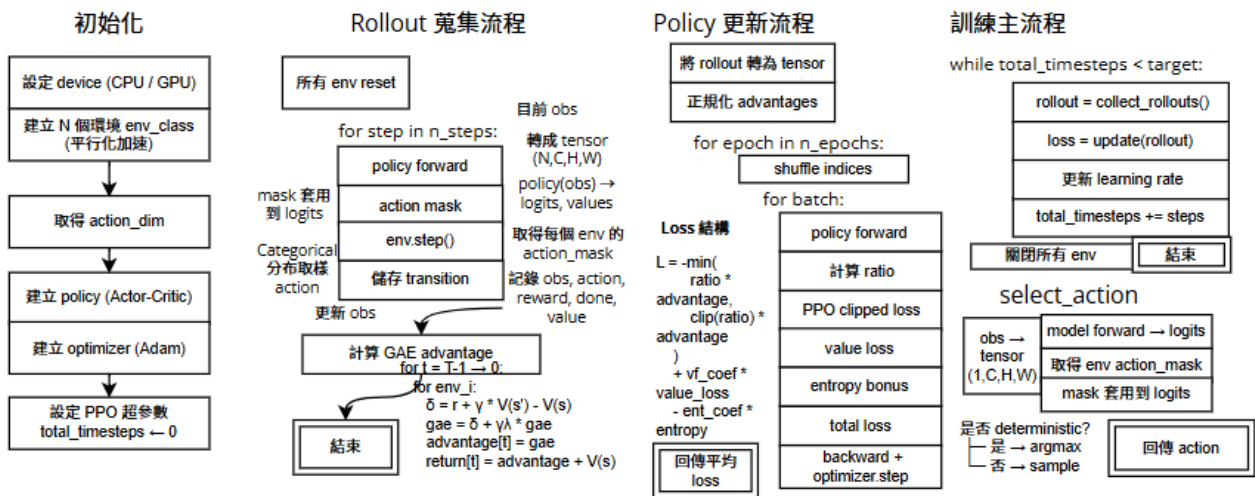
Long Short-Term Memory for Actor-Critic

儘管 CNN 擅長提取單步空間特徵，環境往往是部分可觀測的，即代理人無法直接看到整個環境狀態，而必須依靠經過的觀測與行動歷史來推測完整狀態，序列資訊與歷史依賴性變得關鍵，使得策略模型需要具備某種形式的時間記憶能力。LSTM 具備記憶與遺忘機制的循環神經網路，能捕捉長期依賴關係，使策略網路可利用先前觀測序列判斷當前最優行動。結合 CNN 的空間感知能力與 LSTM 的時間記憶能力，CNN 提取每一時刻視覺特徵，而 LSTM 將此特徵序列整合成上下文感知的隱藏狀態表示，輸出更具時間性與策略相關的行動建議。



Program Architecture

PPO 程式架構：



PPO 會同時建立多個環境實例以形成向量化環境，藉此在每個時間步同步蒐集多條軌跡資料，每一次 rollout 階段中，策略網路會根據當前觀測影像輸出對應的行動機率分佈與狀態價值估計，環境加上行動遮罩用於 policy logits 來降低 Action 複雜性，蒐集完成的 rollout 資料包含觀測、動作、回饋、終止狀態與價值估計，隨後透過 Generalized Advantage Estimation (GAE) 計算優勢函，透過結合一步 temporal-difference 誤差與指數衰減機制，在偏差與變異之間取得平衡，使策略更新在保有穩定性的同時仍能有效利用長期回饋訊息，用剪裁機制來限制新舊策略之間的變化幅度。

gae_lambda：控制優勢估計中未來誤差的累積程度。

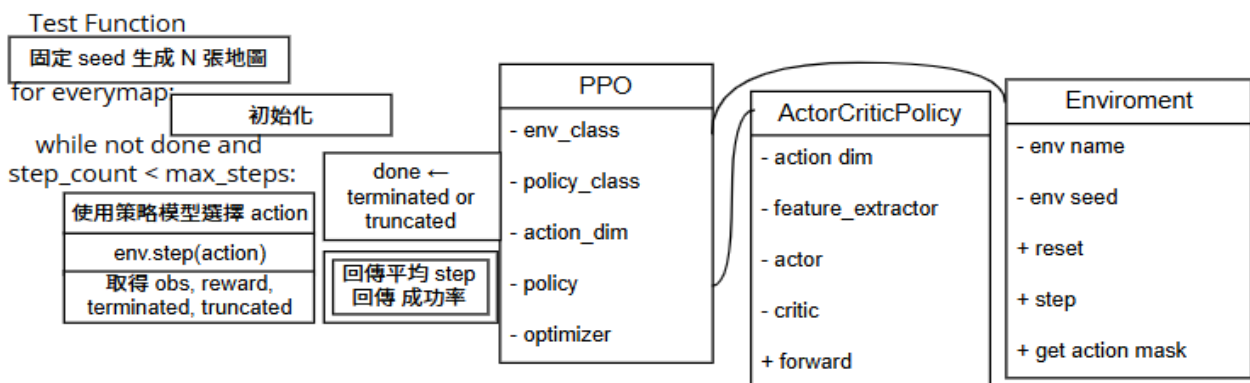
clip_range：定義了機率比值允許偏離 1 的範圍，進而維持訓練過程的穩定。

當 clip_range 設定較小時，策略更新會較為保守

ent_coef：鼓勵策略在訓練初期保持足夠的探索性，避免過早收斂至次佳解。

vf_coef：調整價值函數損失在總損失中的比重，更新時對狀態價值準確性的重視程度。

max_grad_norm：對模型參數的梯度進行裁剪，以有效防止梯度爆炸問題。



Experiment

CNN Policy & PPO

基於實驗使用 PyTorch CNN + PPO 深度強化學習，固定 Seed 測試地圖環境 100 張。

DoorKey - 8 x 8 上限步數：500、DoorKey - 16 x 16 上限步數：1000。

Enviroment	Parallel Envs	RolloutSteps	Train Steps	Test Avg Steps	Test SuccessRate
DoorKey - 8 x 8	8	256	1,501,184	231.2	56.00%
DoorKey - 8 x 8	16	256	2,002,944	54.15	93.00%
DoorKey-16x16	32	512	5,013,504	619.5	40.00%
DoorKey-16x16	32	512	10,010,624	434.1	59.00%
MultiRoom*2	128	512	1,048,576	7.8	99.00%
MultiRoom*4	128	512	1,048,576	68.0	60.00%

從上述圖表發現，將 CNN 作為 PolicyNetwork 使用是可以在 DoorKey 8*8 的地圖取得很好的成效，然而在 16x16 DoorKey 已出現長期依賴問題，觀察實驗後的測試紀錄發現 Agent 明顯的問題就是會在同個地方反覆來回或走回到原本的區域，這意味 PolicyNetwork 並不能判斷哪裡已經去過，缺乏長期規劃、記憶行動的能力，MultiRoom / KeyCorridor 的環境上問題更明顯。

CNN+LSTM Policy & PPO

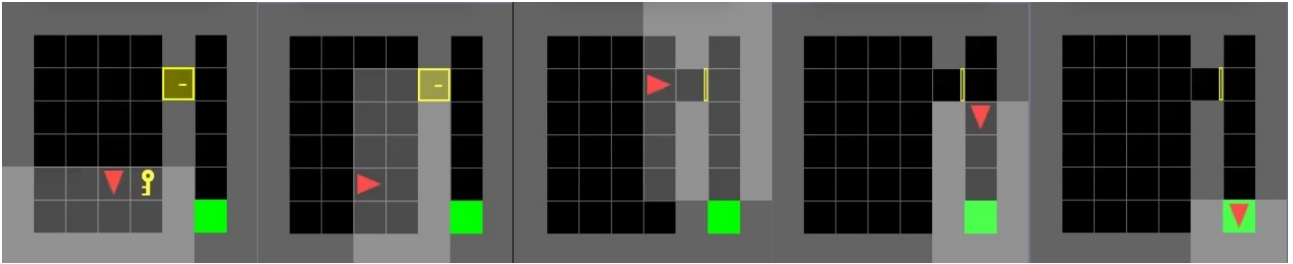
上限步數：500，CNN+LSTM 需要非常長的時間需求訓練..

Enviroment	Parallel Envs	RolloutSteps	Train Steps	Test Avg Steps	Test SuccessRate
DoorKey-16x16	512+	128	25,000,000+	454.1	10.00%
MultiRoom*4	暫時未實驗完				
KeyCorridorS3R1	暫時未實驗完				
KeyCorridorS4R3	暫時未實驗完				

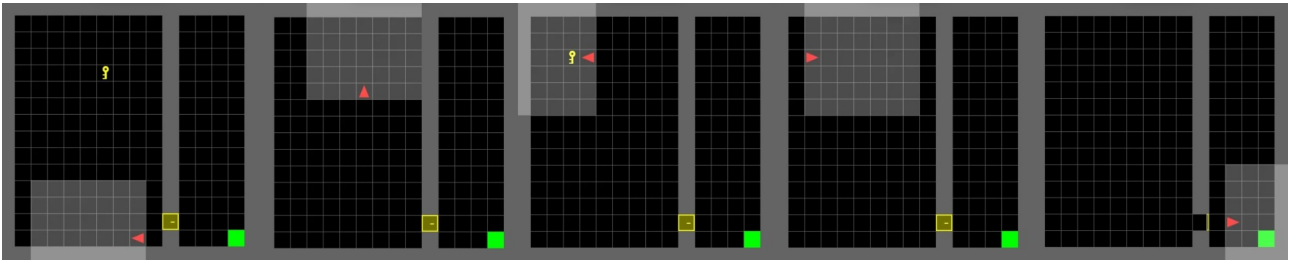
實驗觀察到，CNN+LSTM 架構顯著提高了策略模型的參數量與學習複雜度，訓練樣本必須隨策略更新即時蒐集，樣本利用效率相對有限，需要更長的訓練時間與更多有效軌跡，才能穩定地學習時間記憶，導致尚未收斂，可能引入額外的梯度雜訊，另外，reward 主要集中於少數關鍵事件，中間長時間步驟多為低幅度或負回饋，早期時間步的學習訊號相對微弱，若能未來增加訓練資料量、調整 reward 訊號結構、延長 rollout 序列長度，並更細緻地處理 LSTM 的狀態管理，預期該架構在更複雜與長期依賴的任務中仍具有顯著潛力。

Display & Demo

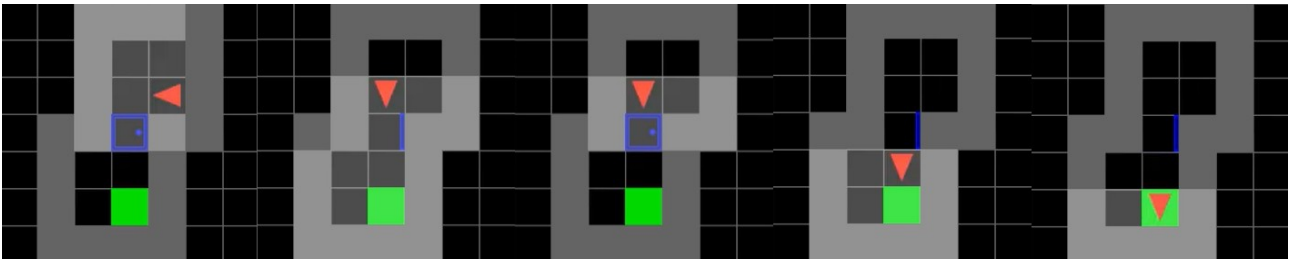
CNN : 8*8 DoorKey



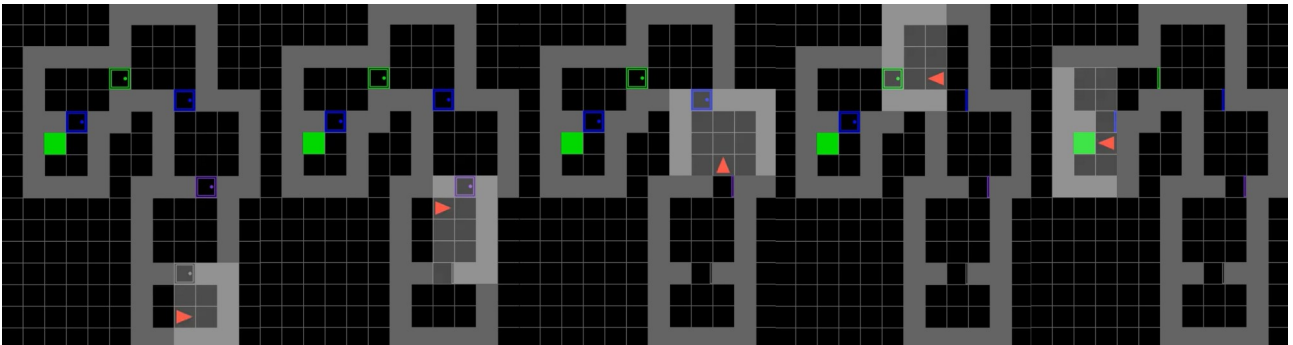
CNN+LSTM : 16*16 DoorKey



CNN : Multi Room * 2



CNN : Multi Room * 4



Github Code :

https://github.com/JiaenSuen/TinyAI/tree/main/SimpleRL/MiniGrid_SpatialNavigation

demo 資料夾內有更多完整 Demo 測試紀錄。

參考文獻

Episodic Novelty Through Temporal Distance , arXiv:2501.15418, 26 Jan 2025

jiang21b-supp , A. Experiment Details and Hyperparameters

結語

目前實驗結論認為在 MiniGrid 這類部分可觀測環境中，通過設計具備空間特徵抽取與時間記憶能力之策略網路，如 CNN 能夠有效處理來自環境的局部視覺輸入，提取關鍵空間資訊可以對環境、空間、幾何的資訊提取物件位置與物件特徵，而後期實驗加入 LSTM 則使策略能夠基於完整的歷史序列，學習並記住先前觀測與行動結果，hidden state 充當 belief state，有助於在部分可觀測任務中做出更精準的決策，將過去的觀測與行為歷史壓縮為內部狀態，使 Actor-Critic 能在缺乏全域資訊的情況下，推斷自身位置、方向與任務進度。

2025 年初時曾使用 MLP 的 DQN 在 CartPole、Mountain Car、Greedy Snake 使用，在理解強化學習後，明白馬可夫過程 DQN 中與強化學習中所講的 State-Action Value 之間的關聯，並在這次真正釐清 DQN 的侷限性與實用方向，並初次瞭解與嘗試挑戰 POMDP Environment 對我來說是新的概念，也是希望能夠藉這次經驗奠定對於自主系統與強化學習算法的初步學習與實驗，實驗中期發現我低估題目的複雜性，迫使我重新檢視環境特性、狀態可觀測性以及 reward 設計是否合理，僅依賴單步觀測進行反應式決策往往不足以支持長期規劃與多階段任務完成，最深刻的體會之一是 reward 設計在強化學習中的關鍵角色，初期沿用 MiniGrid 預設的稀疏獎勵機制時，Agent 幾乎無法獲得有效的學習訊號，導致策略長時間停滯甚至退化，透過逐步加入事件導向的獎勵、距離型 reward shaping 以及行為約束相關的懲罰項目後，Agent 才開始展現出可解釋的行為模式，例如主動搜尋鑰匙、嘗試開門以及朝目標方向移動，其設計品質往往直接決定模型是否具備學習可行性。

為了解決 DQN 的侷限性，才轉向學習並建構 PPO 強化學習算法，嘗試讓 Agent 能在這樣的環境上被訓練，引入 CNN 從局部視覺輸入中有效提取空間結構與物件資訊，在較小地圖（如 DoorKey 8×8）中已能學習出合理且穩定的策略，然而，當環境規模擴大或任務需要長期記憶時，僅依賴 CNN 的策略逐漸暴露出限制，Agent 容易出現重複探索、來回徘徊或遺忘已完成子任務的情況，這也促使我進一步嘗試結合 LSTM，引入時間記憶機制，然而，可惜得訓練成本顯著提高，許多訓練上的問題開始暴露出來，在我所有的實驗設備上根本在短時間內完成，每次新的策略與方法改進實驗都是需要訓練 4~8 小時才會知道可行性，並且許多部份還尚須更多實驗來達成。

儘管本次實驗在部分任務與大尺度環境中尚未達到完全理想的成果，但整體研究已成功建立一套可運作的深度強化學習實驗架構，未來將持續於個人 GitHub 專案中對此系統進行長期維護與精進，逐步優化 PPO 訓練流程、策略網路結構與環境互動設計，實驗較大地圖、多房間結構與長期依賴行為，此外，也計畫嘗試結合自然語言處理模型，使 Agent 能夠根據語言指令或高階任務描述進行行動規劃，藉此提升策略在複雜任務與人機互動場景中的泛化能力，建立強化學習雛形將作為後續機器與自主系統專題的重要基礎，未來將嘗試將模擬環境中學習到的決策策略，與實際感測器資料進行整合，逐步引入真實世界中的感測雜訊、不完整資訊與動態環境因素。

輔助工具：

Grok、ChatGPT 修正 Bug、程式問題，[PlotNeuralNet](#): 視覺化 CNN 架構